

OOP

17. Why do we need to use OOP?

1. making programming clearer and problem-solving more concise
2. reusing code with the help of inheritance
3. reducing redundancy
4. encapsulation
5. data hiding
6. the division into subproblems
7. program flexibility using polymorphism

18. Difference between Overloading and Overriding.

Overloading	Overriding
Overloading means same function name but different parameters.	Overriding means same function name and same parameters.
It happens in the same class.	It happens between parent and child class.
Parameters must be different (number or type).	Parameters must be exactly same.
It is an example of compile-time polymorphism .	It is an example of run-time polymorphism .
Inheritance is not required.	Inheritance is required

19. What is the function of super keyword?

The super keyword is used in inheritance. It is used to refer to the **parent class**.

Main Functions of super:

i. Access parent class variable

If parent and child class have same variable name, super is used to access parent variable.

ii. Call parent class method

It is used to call parent class method from child class.

iii. Call parent class constructor

super () is used to call the constructor of parent class.

20. What is the purpose of “this” keyword?

This keyword is used to refer to the **current class object**.

Main Purposes of “this”:

1. Refer current class variable
2. Call current class method – It can be used to call another method of the same class.
3. Call current class constructor – this () is used to call another constructor of the same class.
4. Pass current object as argument – It can be used to send the current object to another method.

21. What are some major Object-Oriented Programming Languages?

Major object-oriented programming languages include Java, C++, Python, C#, and Ruby.

22. What are some advantages of using Object-Oriented Programming?

Some advantages of object-oriented programming include improved code organization, reusability, and maintainability, as well as the ability to model real-world concepts and encapsulate data and behavior.

1. Code reusability
2. Data Security (Encapsulation)
3. Easy Maintenance
4. Modularity
5. Flexibility (Polymorphism)
6. Real-World Modeling

23. Explain the concept of Inheritance with a real-life example.

Inheritance is a process where one class gets the properties and methods of another class.

Inheritance is a feature of OOP where one class acquires the properties and methods of another class. The parent class provides common features, and the child class inherits those features and can also add new features. For example, Dog inherits common behaviors like eat () from Animal and also has its own behavior like bark ().

Example:

```
Class Animal{
    Void eat(){
        System.out.println("Eating-----");
    }
}
Class Cat extends Animal{
    Void muio(){
        System.out.println("Muio Muio");
    }
}
```

24. Name three Operators that can't be Overloading.

- " :: " Scope resolution operator
- " * " Pointer to member operator
- " . " dot or Member access operator

25. Are there any limitations of inheritance? If yes, then what?

Yes. The limitations of inheritance are:

1. Increased execution effort and time
2. Tight coupling of parent and child class
3. Requires correct implementation
4. Requires jumping between different classes

26. List down the limitations of OOP?

1. Complex Design
2. It requires intensive testing
3. It requires good planning
4. Large Program Size
5. More Memory Usage
6. Not Suitable for Small Programs
7. It takes more time to solve problems

27. Is it always necessary to create objects from class?

No, it is not always necessary to create objects from a class. If a method or variable is static, it can be accessed using the class name without creating an object. Also, abstract classes and interfaces cannot be instantiated directly.

28 . What are the various types of constructors?

- i. Default constructor
- ii. parameterized constructor
- iii. copy constructor
- iv. move constructor

29. Various type of inheritance

- i. Single
- ii. Multiple
- iii. Multi-level
- iv. Hierarchical
- v. Hybrid inheritance

30. (a) What is Subclass?

A subclass is a class that inherits the properties and methods of another class. It is also called a child class. It extends the parent class using inheritance.

(b) What is Superclass?

A superclass is a class that provides properties and methods to another class through inheritance. It is also called the parent or base class.

31. What is meant by static polymorphism?

Static polymorphism, also known as compile-time polymorphism is a form of polymorphism in which the type of an object is determined at compile-time. It is achieved through function overloading, operator overloading and constructor overloading.

32. What is meant by dynamic polymorphism?

Dynamic polymorphism also known as runtime polymorphism, is a form of polymorphism in which the type of an object is determined at runtime. It is achieved through function overriding, Method overriding.

34. How is an abstract class different from an Interface?

An abstract class can contain concrete and pure virtual functions, while an interface can only contain pure virtual parts.

35. What is an exception?

An exception is an abnormal condition or error that occurs during program execution. It interrupts the normal flow of the program. Exceptions can be handled using try-catch blocks to prevent program crash.

36. What is meant by garbage collection in the OOPs world?

Garbage collection is the automatic process of remove unused objects from memory. It frees memory space and prevents memory leaks. In OOP languages like Java, garbage collection is handled automatically by the system.

37. Different abstract class vs Interface.

Abstract class	Interface
1. A class declared with <code>abstract</code> keyword.	1. A blueprint declared with <code>interface</code> keyword.
2. It can have both abstract and normal (concrete) methods.	2. Traditionally, it has only abstract methods (no method body).
3. It can have constructors.	3. It cannot have constructors.
4. It can have instance variables.	4. It has only public static final variables (constants)
5. A class can extend only one abstract class.	5. A class can implement multiple interfaces.